C Programming Notes for Beginners

District Computer Centre, Tinsukia

1. Introduction to C

C is a general-purpose, procedural programming language developed by **Dennis Ritchie** at Bell Labs in 1972. It is widely used for system programming, operating systems, embedded systems, and as a foundation for learning other languages.

Why Learn C?

- Fast and efficient (close to hardware).
- Foundation for C++, Java, Python internals, and operating systems.
- Portable: the same program can run on different machines with little or no change.
- Small standard library that is easy to master.

Structure of a C Program

```
#include <stdio.h>      /* preprocessor directive */

int main(void)          /* main function: program entry point */
{
    printf("Hello, World!\n");   /* statement */
    return 0;           /* return status to the operating system */
}
```

Key parts:

- `#include <stdio.h>` — includes the standard input/output header.
- `main()` — every C program starts executing here.
- `printf()` — prints text to the screen.
- `return 0;` — indicates successful completion.
- Statements end with a semicolon `;`.
- Comments: `/* multi-line */` or `// single line`.

Compilation Process

1. **Writing** the source code in a `.c` file.
2. **Preprocessing** — expands `#include` and `#define` directives.
3. **Compilation** — converts source into assembly/machine code (object file).
4. **Linking** — combines object files and libraries into an executable.

Typical command:

```
gcc hello.c -o hello
./hello
```

---

2. Data Types, Variables, and Constants

Basic Data Types

Type	Typical Size	Range / Use
`int`	4 bytes	Whole numbers, about -2^31 to 2^31-1
`float`	4 bytes	Real numbers, ~6 decimal digits
`double`	8 bytes	Real numbers, ~15 decimal digits
`char`	1 byte	Single character like 'A'
`_Bool` / `int` as flag	1 byte	true (1) / false (0)

Modifiers: `short`, `long`, `signed`, `unsigned` (e.g. `long int`, `unsigned int`).

Variables and Declarations

```
int age = 20;
float price = 99.5f;
char grade = 'A';
double pi = 3.14159;
```

Rules for identifiers (variable names):

- Can contain letters, digits, and underscores.
- Must not start with a digit.
- Cannot be a keyword (e.g. `int`, `return`).
- Case-sensitive: `sum` and `Sum` are different.

Constants

```
const float PI = 3.14f;       /* const keyword */
#define MAX 100              /* preprocessor macro */
```

---

3. Input and Output

Output with printf

```
printf("Sum = %d\n", sum);
```

Common format specifiers:

Specifier	Purpose
`%d` / `%i`	int
`%f`	float / double
`%lf`	double in scanf
`%c`	char
`%s`	string
`%u`	unsigned int
`%x`	hexadecimal
`%p`	pointer address
`%%`	literal %

Width examples: `%5d` (right-aligned in 5 columns), `%.2f` (two decimals), `%-10s` (left-aligned).

Input with scanf

```
 int a;
printf("Enter a number: ");
scanf("%d", &a);          /* & gives the address of a */
```

Note: `scanf` needs the address-of operator `&` for all variables except strings (arrays).

---

4. Operators

- **Arithmetic:** `+ - * / %`
- **Relational:** `== != > < >= <=`
- **Logical:** `&& || !`
- **Assignment:** `= += -= *= /= %=`
- **Increment/Decrement:** `++ --` (pre/post forms)
- **Bitwise:** `& | ^ ~ << >>`
- **Conditional:** `condition ? value1 : value2`

- **sizeof:** returns the size of a type or variable in bytes.

Important notes:

- Integer division truncates: `7 / 2` gives `3`; `7 % 2` gives `1`.
- `=` is assignment; `==` is comparison.
- `&&` and `||` use short-circuit evaluation.

---

5. Control Flow

if / else

```
if (marks >= 40) {
    printf("Pass\n");
} else if (marks >= 30) {
    printf("Re-exam\n");
} else {
    printf("Fail\n");
}
```

switch

```
switch (choice) {
    case 1: printf("One\n"); break;
    case 2: printf("Two\n"); break;
    default: printf("Invalid\n");
}
```

`break` prevents fall-through to the next case.

Loops

```
/* for loop */
for (int i = 1; i <= 5; i++)
    printf("%d ", i);

/* while loop */
int n = 0;
while (n < 5) {
    printf("%d ", n);
    n++;
}

/* do-while: runs at least once */
int k = 0;
do {
    k++;
} while (k < 5);
```

jump statements

- `break` — exits the nearest loop or switch.
- `continue` — skips to the next iteration.
- `return` — exits a function.
- `goto` — jumps to a label (avoid in practice).

---

6. Functions

Functions group reusable logic.

```
int add(int x, int y) {      /* definition */
    return x + y;
}

int main(void) {
    int s = add(3, 4);       /* call */
    printf("%d\n", s);
    return 0;
}
```

- **Declaration (prototype):** `int add(int, int);` placed above `main`.
- **Parameters** are passed by value: a copy of the argument is used.
- **Return type** `void` means nothing is returned.

Function categories: library functions (`printf`, `sqrt`) and user-defined functions.

Recursion: a function calling itself, e.g. factorial:

```
long fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

---

7. Arrays

An array stores multiple values of the same type in contiguous memory.

```
int marks[5] = {80, 75, 90, 60, 85};

for (int i = 0; i < 5; i++)
    printf("%d ", marks[i]);
```

- Indexing starts at 0; the last valid index is the array size minus one.
- Array size must be constant in basic C89-style arrays.
- Arrays are always passed to functions as pointers to the first element.

- No built-in bounds checking — accessing beyond the end is undefined behaviour.

2-D Arrays

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

---

8. Strings

In C, a string is an array of characters terminated by the null character `'\0'`.

```
char name[] = "C Language";     /* size = 10 + 1 */
char city[20];
scanf("%s", city);              /* reads one word */
printf("%s\n", name);
```

Common string functions (`string.h`):

Function	Purpose
`strlen(s)`	length of string
`strcpy(dest, src)`	copy string
`strcmp(a, b)`	compare (0 if equal)
`strcat(a, b)`	concatenate
`strchr(s, c)`	find character

---

9. Pointers

A pointer stores the memory address of a variable.

```
int a = 10;
int *p = &a;        /* p holds the address of a */

printf("%d\n", *p); /* dereferencing: prints 10 */
*p = 20;            /* changes a to 20 through the pointer */
```

- `&` — address-of operator.
- `*` — dereference operator.
- Pointer arithmetic: `p + 1` moves to the next element of the pointed-to type.
- `NULL` pointer — points to nothing; safe to test with `if (p != NULL)`.

- Pointers enable call by reference and dynamic memory.

Call by Reference Example

```
void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}
```

Pointers and Arrays

The array name acts like a pointer to its first element: `arr[i]` is equivalent to `*(arr + i)`.

---

10. Dynamic Memory

Functions from `stdlib.h` allocate memory at runtime from the heap:

```
int *arr = malloc(5 * sizeof(int));   /* allocate */
if (arr == NULL) { /* handle failure */ }

free(arr);                            /* release when done */
arr = NULL;                           /* avoid dangling pointer */
```

- `malloc(size)` — allocate, contents uninitialized.
- `calloc(n, size)` — allocate and zero-initialize.
- `realloc(ptr, size)` — resize.
- `free(ptr)` — release. Every allocation should be freed exactly once.

---

11. Structures

A structure groups different data types under one name.

```
struct Student {
    char name[50];
    int roll;
    float marks;
};

struct Student s1 = {"Asha", 12, 88.5f};
printf("%s %d %.1f\n", s1.name, s1.roll, s1.marks);

struct Student *ptr = &s1;
printf("%s\n", ptr->name);     /* arrow for pointer access */
```

- Access members with the dot operator (`.`) for variables.

- Access members with the arrow operator (`->`) for pointers.
- `typedef` simplifies declarations: `typedef struct Student Student;`

---

12. File Handling

Text files are handled through `FILE *` from `stdio.h`.

Mode	Meaning
`"r"`	read
`"w"`	write (overwrites / creates)
`"a"`	append
`"r+"`	read and write

```
 FILE *fp = fopen("notes.txt", "w");
if (fp == NULL) {
    printf("Cannot open file\n");
    return 1;
}
fprintf(fp, "Welcome to C\n");
fclose(fp);

/* reading back */
char line[100];
fp = fopen("notes.txt", "r");
while (fgets(line, sizeof(line), fp) != NULL)
    printf("%s", line);
fclose(fp);
```

Other functions: `fscanf`, `fgetc`, `fputc`, `fread`, `fwrite` (binary), `fseek`, `ftell`, `rewind`.

---

13. Preprocessor Directives

- `#include <file>` — standard header; `#include "file"` — your own header.
- `#define NAME value` — macro constant.
- `#define SQUARE(x) ((x) * (x))` — macro with parameters.
- `#ifdef`, `#ifndef`, `#endif` — conditional compilation.

Always parenthesize macro parameters to avoid operator-precedence errors.

---

14. Common Errors and Good Practices

Typical Beginner Mistakes

- Forgetting `;` at the end of statements.
- Using `=` instead of `==` in conditions.
- Missing `&` in `scanf`.
- Array index out of bounds.
- Not returning/freeing allocated memory.
- Ignoring the return value of `fopen` / `malloc`.

Good Practices

- Give variables meaningful names.
- Keep functions short — one task per function.
- Indent consistently and comment non-obvious logic.
- Compile with warnings enabled: `gcc -Wall -Wextra code.c`.
- Test with normal, edge, and invalid inputs.

---

15. Suggested Practice Programs

1. Print "Hello, World!" and your name.
2. Add, subtract, multiply, and divide two numbers.
3. Find the largest of three numbers using `if-else`.
4. Print the multiplication table of a given number.
5. Reverse a five-digit number using a loop.
6. Check whether a number is prime.
7. Compute the factorial of a number using recursion.
8. Find the sum and average of ten array elements.
9. Read a string and count vowels.
10. Swap two numbers using pointers.
11. Store and print three student records using structures.
12. Copy the contents of one text file to another.

Quick Reference Card

Concept	Syntax
Include header	`#include <stdio.h>`
Main	`int main(void) { ... return 0; }`
Print	`printf("%d", x);`

Concept	Syntax
Read	`scanf("%d", &x);`
If	`if (cond) { } else { }`
Loop	`for (init; cond; step) { }`
While	`while (cond) { }`
Function	`ret_type name(params) { }`
Array	`int a[10];`
Pointer	`int *p = &a;`
Structure	`struct S { int x; };`
File	`FILE *fp = fopen("f.txt", "r");`

Prepared for beginners at District Computer Centre, Tinsukia. Compile examples with gcc and practise each concept by writing small programs.